

ISO SECURE

Guía Didáctica del Proyecto

Cómo entender lo que hicimos

Material de estudio — técnico pero accesible

Agustín Peralta — Universidad de San Buenaventura

Mayo de 2026

Resumen

Esta guía explica el proyecto **ISO SECURE** en lenguaje claro pero técnico. Está pensada para que cualquier persona con conocimientos básicos de software entienda *qué hicimos, por qué lo hicimos así y cómo verlo funcionando*. Las URLs reales del producto desplegado aparecen al final.

Índice

1. ¿Qué es ISO SECURE? (en 3 párrafos)	2
2. Cómo está construido (la cocina)	2
2.1. Las tres partes principales	2
2.2. Cómo conversan entre sí	2
2.3. Decisiones que tomamos y por qué	3
3. Las 5 etapas del trabajo (qué fuimos haciendo)	3
3.1. Etapa 1 — Requerimientos (qué tiene que hacer el sistema)	3
3.2. Etapa 2 — Arquitectura y Diseño	3
3.3. Etapa 3 — Codificación e Integración	4
3.4. Etapa 4 — Pruebas y Despliegue	4
3.5. Etapa 5 SUMATIVA — Entrega Final (20 %)	4
4. Glosario rápido (palabras técnicas que sí o sí aparecen)	5
5. Cómo está conectado todo (diagrama mental)	5
6. Cumplimiento normativo (lo que la norma dice y cómo lo cumplimos)	6
7. URLs del producto desplegado (¡aquí está todo!)	6
7.1. URL principal de la aplicación	6
7.2. Páginas de documentación y diagramas (todas accesibles públicamente) . .	6
7.3. Repositorio de código	6

8. Cómo estudiar el proyecto en orden	7
9. Cierre	7

1. ¿Qué es ISO SECURE? (en 3 párrafos)

En palabras simples: ISO SECURE es una página web donde una empresa puede llevar el control de su seguridad informática siguiendo la norma internacional **ISO/IEC 27001:2022**. Esa norma es la guía mundial sobre cómo proteger la información: qué controles aplicar, cómo medir incidentes, cómo capacitar al personal, etc.

Para quién: la plataforma sirve a empresas que quieren certificarse en ISO 27001 o que ya lo están y necesitan demostrar que mantienen el sistema vivo. Hay cuatro tipos de usuario: **admin** (gestiona todo), **auditor** (revisa transversalmente), **supervisor** (mira reportes históricos) y **analista** (carga datos del día a día).

Qué ofrece concretamente: el sistema viene cargado con los 93 controles del Anexo A de la norma, un módulo para registrar incidentes de seguridad, una matriz de riesgos, una sección de capacitaciones con videos y un dashboard ejecutivo con KPIs. Cada empresa que se registra ve sólo sus propios datos (**multi-tenant**), aunque corra sobre la misma base.

Resumen visual del producto

Página / con login → Dashboard con métricas → Módulos: Incidentes, Controles ISO, Riesgo, Empresas, Usuarios, Capacitaciones, Implementación, Auditoría.

2. Cómo está construido (la cocina)

2.1. Las tres partes principales

Una aplicación web siempre tiene tres pedazos. ISO SECURE no es la excepción:

1. **Frontend** — lo que el usuario ve en el navegador. Hecho con **React 19** (la librería más popular para interfaces interactivas) y empaquetado con **Vite** (un compilador rápido). Es una *Single Page Application* (SPA): una sola página que cambia su contenido sin recargar el navegador.
2. **Backend** — el cerebro que vive en el servidor. Hecho con **FastAPI**, un framework de Python pensado para construir APIs modernas. El backend expone 39 *endpoints* (puntos de entrada) organizados en 9 routers temáticos: `/auth`, `/incidents`, `/controls`, `/risk`, `/dashboard`, `/empresas`, `/capacitaciones`, `/implementacion`, `/auditoria`.
3. **Base de datos** — donde se guarda la información. Usamos **PostgreSQL 15** hospedado en **Supabase** (un proveedor en la nube). Son 9 tablas: empresas, perfiles de usuario, incidentes, controles, niveles de riesgo, snapshots, cursos, progreso de cursos y ítems de auditoría.

2.2. Cómo conversan entre sí

Flujo de un click típico

1. El usuario hace clic en *"ver incidentes"*.
2. React envía una petición `GET /api/v1/incidents` con su token JWT.
3. FastAPI valida el token contra Supabase Auth.
4. Si es válido, consulta PostgreSQL filtrando por la empresa del usuario.

5. Devuelve la lista en JSON.
6. React la pinta en la pantalla.

2.3. Decisiones que tomamos y por qué

¿Por qué FastAPI y no Django o Flask? FastAPI es asíncrono por defecto (puede manejar muchas peticiones en paralelo) y tiene tipado fuerte vía Pydantic, lo que reduce muchos errores comunes.

¿Por qué Supabase y no una DB propia? Supabase nos da Auth (login, MFA, recuperación de contraseña) ya resuelto, backups automáticos y certificación de seguridad. Acelera el desarrollo y reduce superficie de ataque.

¿Por qué multi-tenant en una sola base? Comparte recursos entre empresas y baja costos. El riesgo (que una empresa vea datos de otra) se mitiga aplicando un filtro estricto por `empresa_id` en cada consulta.

3. Las 5 etapas del trabajo (qué fuimos haciendo)

El proyecto siguió un ciclo formal llamado **SSDLC** (Secure Software Development Lifecycle) en 5 etapas. Cada una con un PDF entregable.

3.1. Etapa 1 — Requerimientos (qué tiene que hacer el sistema)

Antes de escribir una sola línea de código, listamos todo lo que el sistema debía hacer. En total:

- Requisitos de seguridad (Confidencialidad, Integridad, Disponibilidad).
- Requerimientos funcionales (qué hace el sistema: login, listar incidentes, etc.).
- Requerimientos no funcionales (qué tan rápido, qué tan escalable, qué tan mantenible).
- Historias de usuario por rol.
- Casos de uso y *casos de abuso* (cómo alguien podría usarlo mal).

Lección: pensar primero qué se va a construir evita rehacer cosas después.

3.2. Etapa 2 — Arquitectura y Diseño

Acá decidimos *cómo* se iba a construir y modelamos los ataques posibles. Tres componentes clave:

1. **Análisis de riesgos:** identificamos 15 riesgos, los calificamos con probabilidad $\times$ impacto y armamos una matriz de colores (verde/amarillo/rojo). Resultó 1 riesgo crítico (rojo) y 7 altos.
2. **Patrones de diseño:** elegimos cómo organizar el código (capas, inyección de dependencias, factories, etc.) y por qué.

3. **Modelado de ataques:** usamos dos metodologías combinadas, **PASTA** (proceso) + **STRIDE** (taxonomía). Catalogamos 24 amenazas y armamos 3 árboles de ataque (cómo un atacante podría llegar a su objetivo).

¿Qué es STRIDE? (truco para acordarse)

Cada letra es una categoría de amenaza:

Spoofing (suplantar), **T**ampering (modificar), **R**epudiation (negar haber actuado), **I**nformation Disclosure (filtrar), **D**enial of Service (tirar el servicio), **E**levation of Privilege (escalar permisos).

3.3. Etapa 3 — Codificación e Integración

Acá efectivamente *programamos*, pero siguiendo reglas estrictas:

- Mapeamos cada vulnerabilidad común (**OWASP Top 10**) a una práctica concreta del código.
- Usamos un flujo de Git con *Pull Requests* obligatorios y revisión por pares.
- Configuramos **análisis estático (SAST)** automático: herramientas que leen el código y avisan si hay patrones peligrosos (ruff, bandit, semgrep).
- Configuramos **análisis de dependencias (SCA)**: revisa que ninguna librería que usemos tenga vulnerabilidades conocidas (pip-audit, npm audit, Trivy).

Por qué importa: la mayoría de las vulnerabilidades se introducen al codificar. Detectarlas en CI/CD evita que lleguen a producción.

3.4. Etapa 4 — Pruebas y Despliegue

Con el código corriendo, lo atacamos a propósito:

- **Análisis dinámico (DAST):** herramientas como **OWASP ZAP** y **Nuclei** disparan ataques reales contra el sitio y reportan lo que pasa.
- **Configuración segura:** endurecimos cada capa (sistema operativo, Docker, Nginx, base de datos, aplicación, frontend). Por ejemplo: TLS 1.3 obligatorio, cabeceras de seguridad HTTP, contenedores como usuario no-root.
- **Gestión de secretos:** contraseñas y tokens nunca en el código, siempre en un vault con rotación cada 90 días.
- **Despliegue blue-green:** si la nueva versión falla, automáticamente vuelve a la versión anterior.
- **Plan de respuesta a incidentes** con tiempos de respuesta por severidad (P0 = 15 minutos, P3 = 5 días).

3.5. Etapa 5 SUMATIVA — Entrega Final (20 %)

Documento maestro que consolida las 4 etapas anteriores + un ZIP con todos los soportes (PDFs, código, diagramas, anexos ISO).

4. Glosario rápido (palabras técnicas que sí o sí aparecen)

Término	Qué significa en este proyecto
SGSI	Sistema de Gestión de Seguridad de la Información (lo que ISO 27001 pide).
ISO 27001:2022	Norma internacional para gestionar seguridad de la información.
Anexo A	Lista de 93 controles concretos que la norma sugiere implementar.
JWT	Token corto que prueba quién es el usuario sin guardarlo en la sesión del servidor.
RBAC	Control de acceso basado en roles (admin, auditor, supervisor, analista).
Multi-tenant	Una sola aplicación atiende a varias empresas sin que se vean entre sí.
SAST	Análisis estático: leer el código en busca de problemas.
DAST	Análisis dinámico: atacar la app corriendo para encontrar problemas.
SCA	Análisis de dependencias: revisar librerías de terceros.
CI/CD	Integración y despliegue continuo: automatización de pruebas y deploy.
OWASP	Organización mundial que mantiene los catálogos de vulnerabilidades web.
STRIDE	Método para clasificar amenazas en 6 categorías.
PASTA	Proceso de 7 etapas para modelar amenazas alineado al negocio.
TLS 1.3	Versión moderna del cifrado que usa todo HTTPS.
CSP	Cabecera HTTP que limita de qué orígenes puede cargar recursos el navegador.
PITR	Point-in-Time Recovery: restaurar la base de datos a cualquier momento del pasado reciente.
PDCA	Ciclo de mejora continua: Plan-Do-Check-Act. Lo exige ISO 27001.

5. Cómo está conectado todo (diagrama mental)

```

[ Navegador ]
  |
  | HTTPS + JWT
  v
[ Nginx + TLS 1.3 ] <-- reverse proxy con headers de seguridad
  |
  v
[ FastAPI (Python) ]
  |           |
  |           +--> [ Supabase Auth ] (valida cada token)
  |
  v
[ PostgreSQL ] <-- filtro empresa_id en CADA query
  |
  +--> backups diarios + PITR 7 días
  +--> RLS (Row Level Security) activado

[ n8n self-hosted ] <-- webhooks para reportes

```

6. Cumplimiento normativo (lo que la norma dice y cómo lo cumplimos)

ISO 27001:2022 tiene cláusulas obligatorias (4 a 10) y un Anexo A con controles sugeridos. ISO SECURE cubre técnicamente alrededor de 30 controles del Anexo A. Algunos ejemplos:

Control	Qué pide	Cómo lo cumplimos
A.5.15	Control de acceso	RBAC con 4 roles + guards en backend y frontend
A.5.17	Información de autenticación	Supabase Auth con JWT corto + MFA
A.5.24	Planificación de IR	Procedimiento documentado con SLAs por severidad
A.8.2	Privilegios	Principio de mínimo privilegio en cada endpoint
A.8.8	Gestión de vulnerabilidades	SAST + DAST + SCA en CI
A.8.13	Backups	PITR de 7 días + snapshot diario externo
A.8.15	Logging	Logs JSON estructurados con 90 días de retención
A.8.24	Criptografía	TLS 1.3 + JWT firmado RS256
A.8.28	Codificación segura	Mapeo OWASP Top 10 + reglas internas
A.8.29	Pruebas de seguridad	DAST automatizado por release

7. URLs del producto desplegado (¡aquí está todo!)

El sistema completo está desplegado y accesible. **Coolify** (la plataforma que usamos para desplegar) hace deploy automático cada vez que se sube código a la rama `main` del repositorio.

7.1. URL principal de la aplicación

<https://iso-secure.agustinynatalia.site/>

Esta es la página de login. Desde acá se entra a todos los módulos según el rol del usuario.

7.2. Páginas de documentación y diagramas (todas accesibles públicamente)

7.3. Repositorio de código

https://github.com/Agustin1231/iso_secure

Ramas:

- `main` — rama productiva (se despliega automáticamente a Coolify).
- `entrega-final` — rama con todos los PDFs de las 5 etapas y el ZIP de soportes.

URL	Qué contiene
/	SPA principal: login + módulos del SGSI.
/docs.html	Documentación interactiva v2.0: 39 endpoints, 9 routers, modelo de datos.
/arquitectura.html	Arquitectura completa en 7 secciones: capas, backend, frontend, stack, despliegue, autenticación/RBAC, ER.
/threat-modeling.html	Modelado de amenazas STRIDE: 9 secciones, 24 amenazas, árboles de ataque, matriz de riesgo.
/metodologia-amenazas.html	Metodología híbrida PASTA + STRIDE: comparativa de 6 metodologías, 7 etapas PASTA aplicadas.
/manual.html	Manual de usuario con videos por módulo + anexos descargables.

8. Cómo estudiar el proyecto en orden

Para quien quiera entenderlo bien, este es el camino recomendado:

1. Leer esta guía completa (estás en eso ahora mismo).
2. Abrir [/arquitectura.html](#) y recorrer las 7 secciones.
3. Ver [/threat-modeling.html](#) para ver cómo se identifican amenazas.
4. Leer el PDF **etapa-2-arquitectura.pdf** (es el más conceptual).
5. Si interesa la parte normativa: **etapa-5-entrega-final.pdf**, sección de cumplimiento.
6. Si interesa la parte técnica: **etapa-3-codificacion.pdf** (OWASP, SAST) y **etapa-4-pruebas-despliegue.pdf** (DAST, hardening).
7. Para profundizar en el código: clonar el repo de GitHub y leer `CONTEXT.md` + `README.md`.

Importante para los evaluadores

La aplicación corre con base de datos Supabase. Para hacer pruebas en producción se requiere un usuario válido (registro habilitado o credencial entregada por el autor). Las páginas [/arquitectura.html](#), [/threat-modeling.html](#), [/metodologia-amenazas.html](#), [/docs.html](#) y [/manual.html](#) son **públicas** y no requieren login.

9. Cierre

Este proyecto demuestra que es posible construir software pensando en seguridad desde el primer día, no como un parche al final. El ciclo SSDLC en 5 etapas nos obligó a:

- Definir bien qué se quería antes de programar.
- Modelar las amenazas antes de exponerse a ellas.
- Automatizar la detección de problemas en lugar de confiar en revisión manual.

- Atacar nuestro propio sistema antes que un externo.
- Documentar todo en formato auditable.

El resultado: una plataforma funcional, cumpliendo ~30 controles del Anexo A de ISO 27001:2022, desplegada y accesible, con todo el rastro técnico y documental disponible públicamente.